

1. Introduction

For this project, I developed a Wordle game using Python. The goal of the game is for the player to guess a hidden five-letter word within five attempts. After each guess, the game provides feedback using colored boxes. A green box indicates that a letter is in the correct position, a yellow box indicates that the letter exists in the word but is in the wrong position, and a white box indicates that the letter is not present in the word.

Some of the key features of the game are:

- Random selection of a target word from a list.
- Input validation to ensure the player enters a five-letter word.
- Feedback after each guess using colored boxes.
- Multiple rounds of gameplay.
- Tracking of player statistics, including games played, games won, points earned, and win rate.
- Display of previous guesses and feedback during each round.
- Object-oriented design using a class.

2. Design and Implementation

Before beginning the implementation, I created a basic design for how the game should function. The game loop would select a random word, allow the player to make guesses, compare the guesses to the target word, and provide feedback until the player either won or ran out of attempts.

Firstly, I programmed the game using only variables and loops. The first version had a list of words, a random target word, and a loop that allowed the player to enter guesses. Once the basic gameplay was working, I started adding more features.

One of the first design decisions I made was to store the possible words inside a Python list. This allowed me to easily select a random word using the `random.choice()` function. I also used a list to store the player's previous guesses throughout a round.

As the project was getting bigger, I decided to reorganize the code using a class called `WordleGame`. This helped separate the game's functionality into smaller methods with specific roles. For example:

- `print_header()` displays the instructions and game title.
- `generate_feedback()` compares the guess with the target word.
- `play_round()` handles the gameplay for a single round.
- `show_stats()` displays player statistics.
- `start_game()` controls the overall game loop.

One challenge I faced was deciding how to display feedback to the player. In the beginning, I was going to use letters like G, Y, and _ to represent feedback. However, I chose to use colored Wordle-style boxes instead because they are more intuitive and make the game easier to understand.

I also modified the original design by adding a statistics system. The game now tracks the number of games played, games won, total points, and win percentage. This feature was not part of the original concept but improved the user experience and provided additional practice with variables and calculations.

```
=====
WORDLE
=====

How to play:
- Guess the hidden 5-letter word
- You have 5 attempts per round
- Feedback after each guess:
  ■ correct letter, correct position
  ■ correct letter, wrong position
  ■ letter not in the word

Try to guess the word before attempts run out.
=====

Previous guesses:

Attempts remaining: 5
Enter a 5-letter word: █

=====
STATISTICS
=====
Games Played: 2
Games Won: 2
Points: 20
Win Rate: 100.0%
=====

Press Enter to continue...█

=====
STATISTICS
=====
Games Played: 3
Games Won: 3
Points: 30
Win Rate: 100.0%
=====

Press Enter to continue...█

=====
WORDLE
=====

How to play:
- Guess the hidden 5-letter word
- You have 5 attempts per round
- Feedback after each guess:
  ■ correct letter, correct position
  ■ correct letter, wrong position
  ■ letter not in the word

Try to guess the word before attempts run out.
=====

Correct word!
You used 2 attempts.
Points earned: 10

Press Enter to continue...█

=====
STATISTICS
=====
Games Played: 2
Games Won: 2
Points: 20
Win Rate: 100.0%
=====

Press Enter to continue...
Play another round? (y/n): █
```

3. Conclusions

This project helped me strengthen my understanding of Python programming and object-oriented design. Although I had previous programming experience from college courses,

this project gave me practical experience applying concepts such as classes, methods, loops, conditionals, lists, and user input in a complete application.

One of the most valuable lessons I learned was the importance of organizing code. As the project became larger, it became clear that placing everything inside one large loop would make the code difficult to maintain. Creating a class and dividing the program into methods made the project much easier to understand and expand.

I believe the strongest features of the project are the clean user interface, the Wordle-style feedback system, the statistics tracking, and the ability to play multiple rounds without restarting the program. These features make the game more engaging and provide a better user experience.

The main shortcoming of the project is that it uses a relatively small list of words. Also, the game currently accepts any five-letter input rather than checking against a large dictionary of valid English words.

If I were to redesign the project, I would likely separate the statistics system into its own class rather than keeping everything inside the WordleGame class. This would improve the separation of roles and make the design more scalable.

Some features that I want to add to the program later are:

- A larger dictionary of valid words.
- Difficulty levels.
- Saving statistics to a file.
- A graphical user interface.